Programare Web / Dezvoltarea Aplicațiilor Web
Tehnologia Informației, anul IV · Informatică, anul II · Anul universitar 2026–2027, semestrul I

LABORATORUL 07

Baza de date: entități, relații și migrări

Adăugăm în aplicație entitatea Review (recenzia unui produs), o legăm de Product și o ducem până în tabela din PostgreSQL.

Tema	Tema 9 din planul disciplinei (examene.pe.ub.ro): baze de date relaționale, ORM și migrări
Durata	2 ore
Aveți nevoie de	proiectul `daw-template-ti-info-2026-2027` funcțional (laboratoarele 01–06), Docker Desktop, editorul de cod și DBeaver
La final aveți	entitatea `Review`, migrarea ei rulată, tabela `review` cu câteva recenzii și un commit pe branch-ul vostru

1. Obiective

La finalul laboratorului veți putea:

- explica ce sunt o tabelă, o cheie primară și o cheie străină și cum arată o relație unu-la-mulți (*one-to-many*);
- genera o entitate Doctrine cu `make:entity`, inclusiv un câmp de tip relație `ManyToOne`;
- crea, citi și rula o migrare care modifică structura bazei de date;
- verifica în DBeaver tabela nouă, insera date cu `INSERT` și le citi cu un `SELECT` care folosește `JOIN`.

2. Pe scurt: date relaționale și ORM

O **bază de date relațională** păstrează datele în **tabele**: fiecare rând este o înregistrare, fiecare coloană un atribut [1], [4]. Fiecare rând are o **cheie primară** (`id`), unică. Când un rând se referă la un rând din altă tabelă, păstrăm id-ul acestuia într-o coloană numită **cheie străină** (*foreign key*), iar baza de date verifică singură ca acel id să existe.

Un produs are **multe** recenzii, o recenzie aparține unui **singur** produs: relație unu-la-mulți. Cheia străină stă pe partea „mulți”: tabela `review`, coloana `product_id`. Din punctul de vedere al recenziei, relația se numește `ManyToOne` (multe recenzii → un produs) [3].

În Symfony, **Doctrine ORM** leagă o clasă PHP (o **entitate**) de o tabelă: proprietățile devin coloane, obiectele devin rânduri [2]. Schimbările de structură se salvează în **migrări** (fișiere PHP cu comenzile SQL), care ajung în Git; orice coleg își aduce astfel baza de date la aceeași structură.

3. Pornirea

> **Unde.** Comenzile se scriu în terminalul editorului, în folderul `daw-template-ti-info-2026-2027`. Pentru instalare și probleme frecvente, vedeți „Metodologie de lucru și template de proiect” [5].

1. Porniți **Docker Desktop** și așteptați mesajul **Engine running**. Containerele `daw-nginx`, `daw-php`, `daw-postgres` și `daw-react` trebuie să ajungă în starea **Running**.
2. Verificați pe ce branch sunteți: `git branch` marchează ramura curentă cu `*` (sau `git status`, pe primul rând: `On branch …`). Doar dacă nu sunteți deja pe branch-ul vostru, treceți pe el; apoi porniți containerele (înlocuiți `popescu.ion`):

```
git branch                # ramura curenta are * in fata
# doar daca NU sunteti deja pe branch-ul vostru:
git checkout popescu.ion   # sau: git switch popescu.ion
docker compose -f backend/compose.yaml up -d
docker compose -f frontend/compose.yaml up -d
```

4. Entitatea Review cu make:entity

1. Porniți generatorul. El scrie clasa entității și clasa repository (cea care citește din tabelă) și rulează în containerul `daw-php`, unde există PHP și Symfony:

   ```
   docker exec -it daw-php php bin/console make:entity Review
   ```

2. Generatorul întreabă, pe rând, numele fiecărui câmp (`New property name`), tipul lui (`Field type`) și dacă poate lipsi (`nullable`). Răspundeți ca în tabel. **Enter** înseamnă că acceptați valoarea dintre parantezele drepte; la orice întrebare în plus (de exemplu despre *broadcast*), apăsați Enter.

Câmp (New property name)	Tip (Field type)	Celelalte întrebări
`author` – numele autorului	Enter (`string`)	`Field length`: `100`; nullable: Enter (`no`)
`rating` – nota	`integer`	nullable: Enter (`no`)
`comment` – textul recenziei	`text` (text lung)	nullable: `yes` (comentariul e opțional)
`createdAt` – data	Enter (se propune `datetime_immutable`)	nullable: Enter (`no`)
`product` – produsul	`relation`	întrebările de mai jos

Pentru câmpul `product` urmează întrebările despre relație. Înainte de tipul relației, generatorul afișează un tabel care explică fiecare variantă; merită citit.

- `What class should this entity be related to?` → `Product` (recenzia se referă la un produs);
- `Relation type? [ManyToOne, OneToMany, ManyToMany, OneToOne]` → `ManyToOne` (multe recenzii, un produs);
- `Is the Review.product property allowed to be null (nullable)?` → `no` (orice recenzie are un produs);
- `Do you want to add a new property to Product …?` → `no` (nu modificăm acum clasa `Product`);
- la următorul `New property name` apăsați Enter. Apare **Success!** și două fișiere noi în `backend/app/src/`: `Entity/Review.php` și `Repository/ReviewRepository.php`.

4.1 Codul complet al entității

Înlocuiți conținutul fișierului `Review.php` cu varianta de mai jos. Față de ce a generat comanda, sunt **trei adăugiri**: `onDelete: 'CASCADE'` (ștergerea unui produs îi șterge și recenziile), constructorul (completează data) și metoda `toArray()`, după modelul entităților `Product` și `Category`. Getter-ele sunt scrise pe scurt, iar operatorul `?->` întoarce `null` dacă obiectul lipsește.

```
<?php

namespace App\Entity;

use App\Repository\ReviewRepository;
use Doctrine\DBAL\Types\Types;
use Doctrine\ORM\Mapping as ORM;
```

```php
#[ORM\Entity(repositoryClass: ReviewRepository::class)]
class Review
{
  #[ORM\Id]
  #[ORM\GeneratedValue]
  #[ORM\Column]
  private ?int $id = null;

  #[ORM\Column(length: 100)]
  private ?string $author = null;

  #[ORM\Column]
  private ?int $rating = null;

  #[ORM\Column(type: Types::TEXT, nullable: true)]
  private ?string $comment = null;

  #[ORM\Column]
  private ?\DateTimeImmutable $createdAt = null;

  #[ORM\ManyToOne]
  #[ORM\JoinColumn(nullable: false, onDelete: 'CASCADE')]
  private ?Product $product = null;

  public function __construct()
  {
    $this->createdAt = new \DateTimeImmutable();
  }

  public function getId(): ?int { return $this->id; }
  public function getAuthor(): ?string { return $this->author; }
  public function getRating(): ?int { return $this->rating; }
  public function getComment(): ?string { return $this->comment; }
  public function getCreatedAt(): ?\DateTimeImmutable { return $this->createdAt; }
  public function getProduct(): ?Product { return $this->product; }

  public function setAuthor(string $author): static
  {
    $this->author = $author;
    return $this;
  }

  public function setRating(int $rating): static
  {
    $this->rating = $rating;
    return $this;
  }

  public function setComment(?string $comment): static
  {
    $this->comment = $comment;
    return $this;
  }

  public function setProduct(?Product $product): static
  {
    $this->product = $product;
    return $this;
  }
```

```
  public function toArray(): array
  {
    return [
      'id' => $this->id,
      'author' => $this->author,
      'rating' => $this->rating,
      'comment' => $this->comment,
      'createdAt' => $this->createdAt?->format('Y-m-d H:i'),
      'productId' => $this->product?->getId(),
      'productName' => $this->product?->getName(),
    ];
  }
}
```

5. Migrarea: din clasă în tabelă

5.1 Generarea și citirea migrării

1. Cereți lui Doctrine să compare entitățile cu baza de date și să scrie diferența într-o migrare:
   ```
   docker exec -it daw-php php bin/console doctrine:migrations:diff
   ```
2. Deschideți fișierul indicat în terminal, din `backend/app/migrations/` (numele conține data și ora, de exemplu `Version20261112153000.php`). Citiți metoda `up()`:

```
$this->addSql('CREATE TABLE review (id INT GENERATED BY DEFAULT AS IDENTITY NOT NULL,
  author VARCHAR(100) NOT NULL, rating INT NOT NULL, comment TEXT DEFAULT NULL,
  created_at TIMESTAMP(0) WITHOUT TIME ZONE NOT NULL, product_id INT NOT NULL,
  PRIMARY KEY (id))');
$this->addSql('CREATE INDEX IDX_... ON review (product_id)');
$this->addSql('ALTER TABLE review ADD CONSTRAINT FK_... FOREIGN KEY (product_id)
  REFERENCES product (id) ON DELETE CASCADE ...');
```

Câmpul `createdAt` a devenit coloana `created_at`, iar relația a devenit coloana `product_id` plus o constrângere `FOREIGN KEY` către `product (id)`. Metoda `down()` face operația inversă. Textul exact (sufixele `IDX_`/`FK_`) poate diferi puțin.

> **Atenție.** Dacă migrarea conține și alte tabele decât `review`, înseamnă că migrările vechi nu au fost rulate. Ștergeți fișierul nou, rulați întâi `doctrine:migrations:migrate` (secțiunea 5.2), apoi refaceți `diff`.

5.2 Rularea migrării

1. Rulați migrarea și confirmați cu Enter (`yes`):
   ```
   docker exec -it daw-php php bin/console doctrine:migrations:migrate
   ```
2. Verificați că entitățile și baza de date sunt sincronizate:
   ```
   docker exec -it daw-php php bin/console doctrine:schema:validate
   ```

> **Rezultat.** `migrate` afișează `[OK] Successfully migrated…`, iar `schema:validate` are ambele rânduri verzi (*mapping correct*, *in sync*).

6. Verificare și date de test în DBeaver

1. În DBeaver: **Database → New Database Connection → PostgreSQL → Next**. Completați **Host** `localhost`, **Port** `5432`, **Database** `daw`, **Username** `daw`, **Password** `daw`. Apăsați **Test Connection** (prima dată DBeaver descarcă driverul), apoi **Finish**.
2. Deschideți **daw → Schemas → public → Tables → review** (apăsați **F5** dacă nu apare). Tab-ul **Diagram** arată legătura cu `product`.

3. Click dreapta pe conexiune → **SQL Editor → New SQL script**. Rulați `SELECT id, name FROM product;` cu **Ctrl + Enter** (cursorul pe interogare) și notați id-ul unui produs.
4. Inserați două recenzii (înlocuiți `1` cu id-ul notat), apoi citiți-le: `JOIN` aduce lângă fiecare recenzie numele produsului.

```
INSERT INTO review (author, rating, comment, created_at, product_id)
VALUES ('Ana Popescu', 5, 'Foarte mulțumită, recomand.', NOW(), 1),
       ('Mihai Ionescu', 3, NULL, NOW(), 1);
SELECT r.id, r.author, r.rating, r.comment, p.name AS produs
FROM review r
JOIN product p ON p.id = r.product_id
ORDER BY r.id;
```

7. Salvarea lucrului și oprirea

Verificați ce s-a schimbat, faceți commit și push, apoi opriți containerele:

```
git status
git add .
git commit -m "Laboratorul 07: entitatea Review, ReviewRepository, migrarea"
git push origin popescu.ion
# sau doar: git push (dupa ce ati facut o data git push -u)
docker compose -f backend/compose.yaml stop
docker compose -f frontend/compose.yaml stop
```

> **Push.** `git push origin popescu.ion` este forma completă: trimite branch-ul vostru la `origin`, adresa proiectului de pe GitHub. Forma scurtă, `git push`, merge doar dacă ați făcut anterior, o dată, `git push -u origin popescu.ion`; `-u` leagă branch-ul local de cel de pe GitHub.

8. Exerciții

1. Adăugați din DBeaver încă patru recenzii, pentru cel puțin două produse diferite, cu note diferite.
2. Scrieți o interogare care afișează, pentru fiecare produs, numărul de recenzii și nota medie. Indiciu: `LEFT JOIN`, `COUNT(r.id)`, `ROUND(AVG(r.rating), 2)` și `GROUP BY p.id, p.name`. De ce `LEFT JOIN` și nu `JOIN`?
3. ★ (opțional) Inserați o recenzie cu `product_id` = `9999` (un produs inexistent). Ce mesaj primiți și ce componentă a bazei de date l-a produs?

9. Ce arătați la final

- fișierul `Review.php` și migrarea, explicând `CREATE TABLE review` și `FOREIGN KEY`;
- în DBeaver: rezultatul interogării cu `JOIN` și interogarea de la exercițiul 2;
- commit-ul „Laboratorul 07” pe GitHub, pe branch-ul vostru.

Resurse pentru exersare

- **W3Schools – SQL (cu editor „Try it Yourself”):** https://www.w3schools.com/sql/
- **DBeaver – tutorial video (în engleză):** https://www.youtube.com/watch?v=GgODmMBVUD4
- **Symfony – Databases and the Doctrine ORM:** https://symfony.com/doc/current/doctrine.html
- **PostgreSQL – constrângeri (chei primare și străine):** https://www.postgresql.org/docs/current/ddl-constraints.html

Bibliografie

[1] E. Pruteanu și I. Furdu, *Modele, tehnologii, servicii și aplicații integrate de tip R.I.A. pentru web*. Bacău: Alma Mater, 2022.

[2] Symfony, „Databases and the Doctrine ORM”, documentație oficială. symfony.com/doc/current/doctrine.html

[3] Doctrine Project, „Association Mapping”, *Doctrine ORM Documentation*. doctrine-project.org

[4] PostgreSQL Global Development Group, „Constraints”, *PostgreSQL Documentation*. postgresql.org/docs/current

[5] C.-M. Papară, *Metodologie de lucru și template de proiect – Programare Web / Dezvoltarea Aplicațiilor Web*. Universitatea „Vasile Alecsandri” din Bacău, 2026.